



Assignment-8

Student Name: Anjali

Branch: BE- CSE

Semester: 6

Subject Name: AP LAB-II

UID: 22BCS10665

Section/Group: 22BCS_IOT-614/B

Date of Performance: 10-04-25

Subject Code: 22CSP-351

PROBLEM-1: Maximum Units on a Truck

- **Objective:** You are assigned to put some amount of boxes onto one truck. You are given a 2D array `boxTypes`, where `boxTypes[i] = [numberOfBoxesi, numberOfUnitsPerBoxi]`:

`numberOfBoxesi` is the number of boxes of type `i`.

`numberOfUnitsPerBoxi` is the number of units in each box of the type `i`.

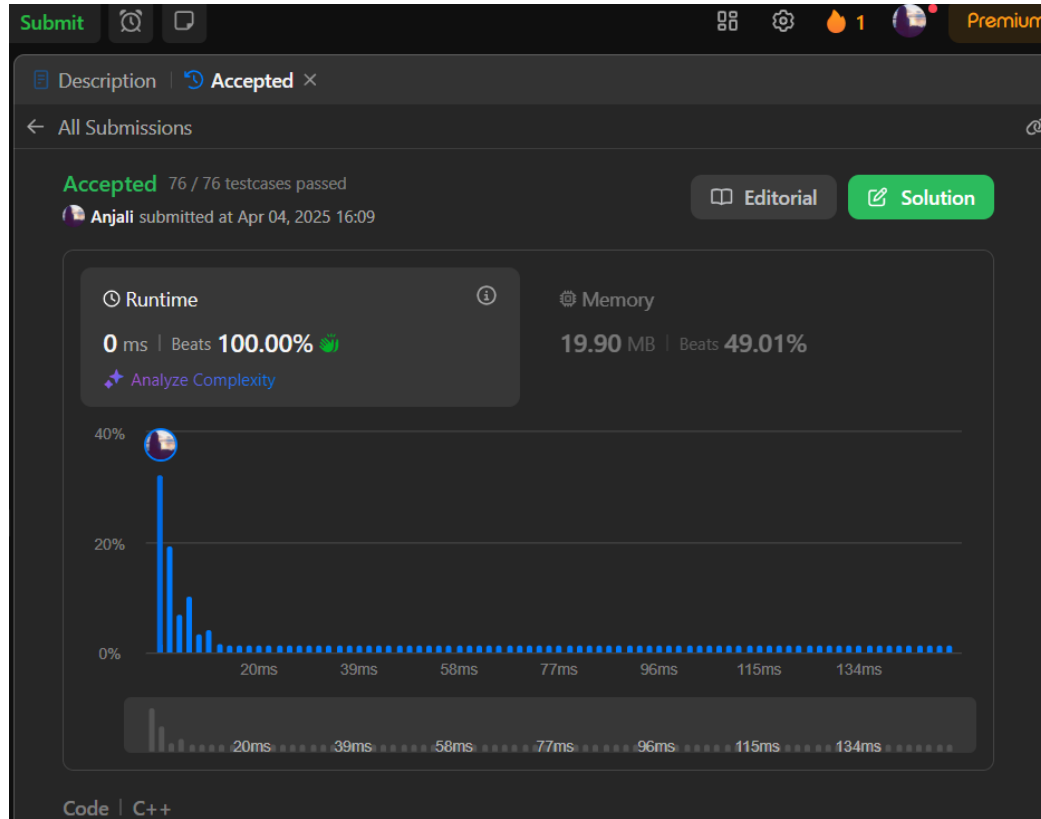
You are also given an integer `truckSize`, which is the maximum number of boxes that can be put on the truck. You can choose any boxes to put on the truck as long as the number of boxes does not exceed `truckSize`.

Return the maximum total number of units that can be put on the truck.

- **Implementation/Code:**

```
class Solution {
public:
    int maximumUnits(vector<vector<int>>& boxTypes, int truckSize) {
        sort(boxTypes.begin(), boxTypes.end(), [](vector<int>& a, vector<int>& b) {
            return a[1] > b[1];
        });
        int maxUnits = 0;
        for (auto& box : boxTypes) {
            int boxCount = min(truckSize, box[0]);
            maxUnits += boxCount * box[1];
            truckSize -= boxCount;
            if (truckSize == 0) break;
        }
        return maxUnits;
    }
};
```

➤ Output:



PROBLEM-2: Maximum Number of Tasks You Can Assign

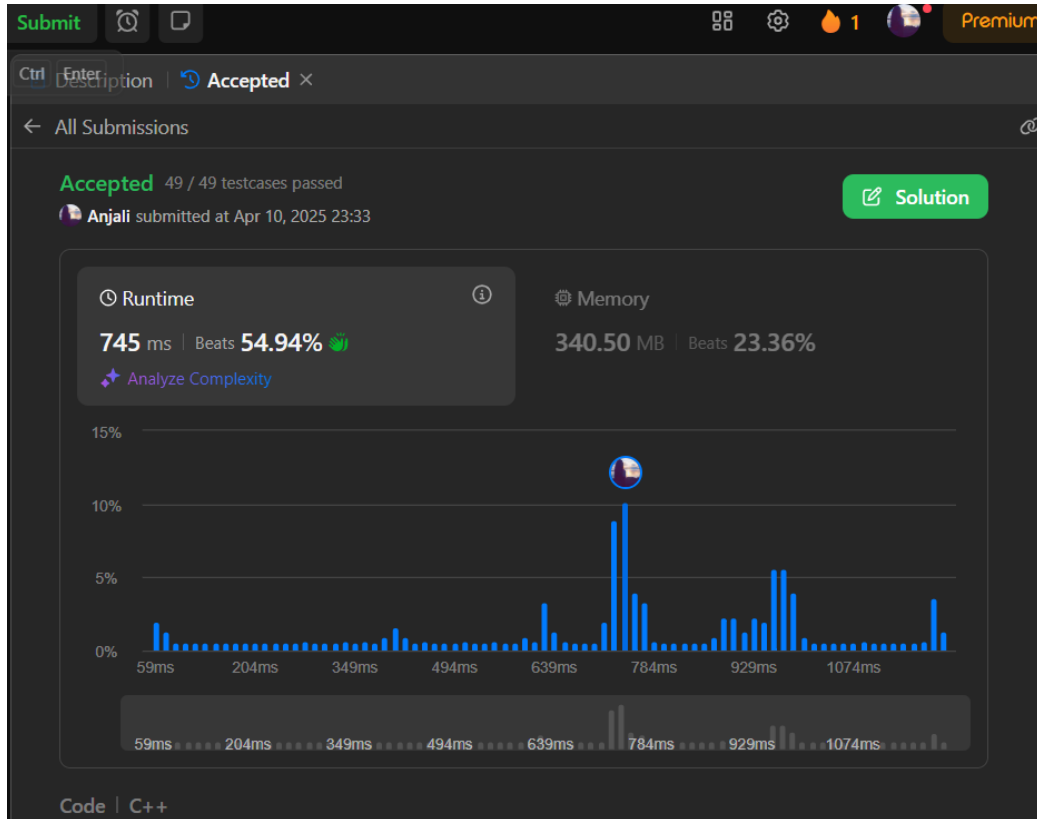
- **Objective:** You have n tasks and m workers. Each task has a strength requirement stored in a 0-indexed integer array `tasks`, with the i^{th} task requiring `tasks[i]` strength to complete. The strength of each worker is stored in a 0-indexed integer array `workers`, with the j^{th} worker having `workers[j]` strength. Each worker can only be assigned to a single task and must have a strength greater than or equal to the task's strength requirement (i.e., `workers[j] >= tasks[i]`). Additionally, you have pills magical pills that will increase a worker's strength by `strength`. You can decide which workers receive the magical pills, however, you may only give each worker at most one magical pill. Given the 0-indexed integer arrays `tasks` and `workers` and the integers `pills` and `strength`, return the maximum number of tasks that can be completed.

➤ **Implementation/Code:**

```
class Solution {
public:
    int maxTaskAssign(vector<int>& tasks, vector<int>& workers, int p, int strength) {
        int n = tasks.size(), m = workers.size();
        sort(tasks.begin(), tasks.end());
        sort(workers.begin(), workers.end());
        int lo = 0, hi = min(m, n);
        int ans;

        while(lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            int count = 0;
            bool flag = true;
            multiset<int> st(workers.begin(), workers.end());
            for(int i = mid - 1; i >= 0; i--) {
                auto it = prev(st.end());
                if(tasks[i] <= *it) {
                    st.erase(it);
                } else {
                    auto it = st.lower_bound(tasks[i] - strength);
                    if(it != st.end()) {
                        count++;
                        st.erase(it);
                    } else {
                        flag = false;
                        break;
                    }
                }
            }
            if(count > p) {
                flag = false;
                break;
            }
        }
        if(flag) {
            ans = mid;
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    return ans;
};
```

➤ Output:



PROBLEM-3: Minimum Operations to Make a Subsequence

- **Objective:** You are given an array target that consists of distinct integers and another integer array arr that can have duplicates.

In one operation, you can insert any integer at any position in arr. For example, if arr = [1,4,1,2], you can add 3 in the middle and make it [1,4,3,1,2]. Note that you can insert the integer at the very beginning or end of the array.

Return the minimum number of operations needed to make target a subsequence of arr.

A subsequence of an array is a new array generated from the original array by deleting some elements (possibly none) without changing the remaining elements' relative order. For example, [2,7,4] is a subsequence of [4,2,3,7,2,1,4] (the underlined elements), while [2,4,2] is not.

- **Implementation/Code:**

```
class Solution {
public:
    int minOperations(vector<int>& target, vector<int>& arr) {
        unordered_map<int, int> pos;
```

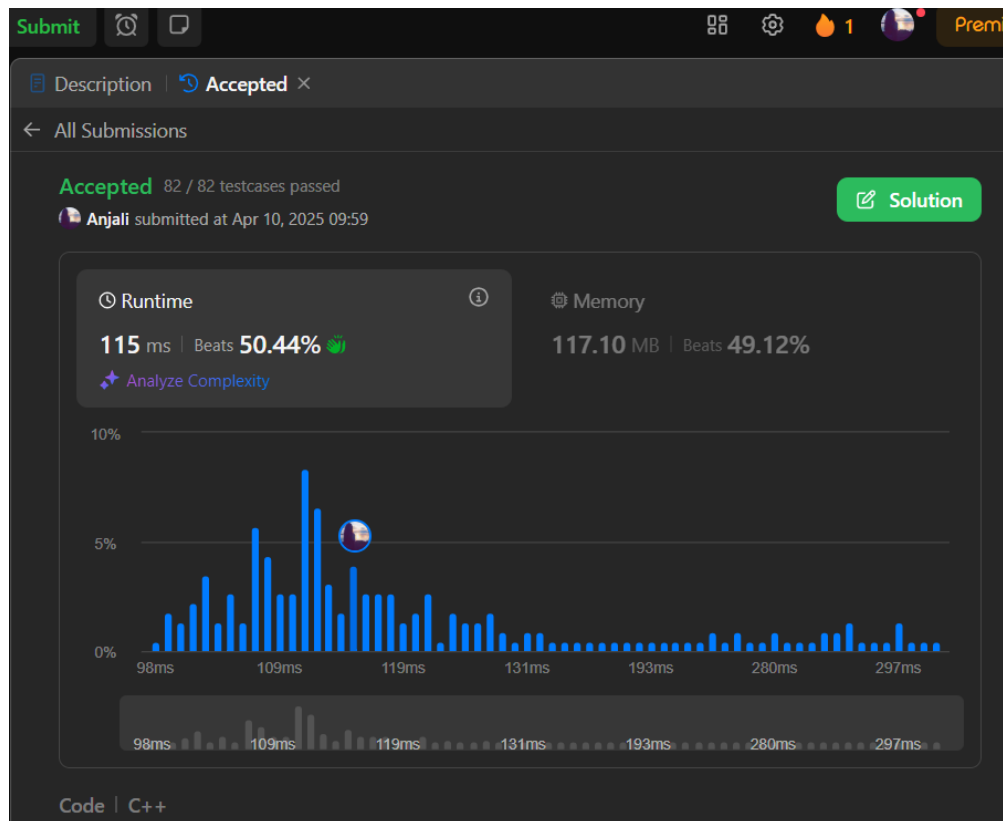


DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
for (int i = 0; i < target.size(); ++i) {  
    pos[target[i]] = i;  
}  
vector<int> indexSeq;  
for (int val : arr) {  
    if (pos.count(val)) {  
        indexSeq.push_back(pos[val]);  
    }  
}  
vector<int> lis;  
for (int num : indexSeq) {  
    auto it = lower_bound(lis.begin(), lis.end(), num);  
    if (it == lis.end()) {  
        lis.push_back(num);  
    } else {  
        *it = num;  
    }  
}  
return target.size() - lis.size();  
}  
};
```

➤ **Output:**



PROBLEM-4: Remove Stones to Minimize the Total

- **Objective:** You are given a 0-indexed integer array `piles`, where `piles[i]` represents the number of stones in the i^{th} pile, and an integer `k`. You should apply the following operation exactly `k` times:
Choose any `piles[i]` and remove $\text{floor}(\text{piles}[i] / 2)$ stones from it.
Notice that you can apply the operation on the same pile more than once.
Return *the minimum possible total number of stones remaining after applying the `k` operations*.
 $\text{floor}(x)$ is the greatest integer that is smaller than or equal to x (i.e., rounds x down).

➤ **Implementation/Code:**

```
class Solution {
public:
    int minStoneSum(vector<int>& piles, int k) {
        priority_queue<int> pq(piles.begin(), piles.end());

        while (k-- > 0) {
            int maxPile = pq.top();
            pq.pop();
            maxPile -= maxPile / 2;
            pq.push(maxPile);
        }
        int total = 0;
        while (!pq.empty()) {
            total += pq.top();
            pq.pop();
        }

        return total;
    }
};
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

➤ **Output:**

